UNIVERSITÀ DI PARMA

Dipartimento di Scienze Matematiche, Fisiche e Informatiche

*Corso di Laurea Triennale in Informatica*

# Titolo in Italiano

Candidato:
**Saverio Mattia Merenda**

Matricola:

Relatore:
**Prof. Vincenzo Arceri**

Anno Accademico 2023–2024

*A me medesimo.*

# Indice

**1 Introduzione** **1**

**2 Background** **3**
2.1 Blockchain . . . . . 3
2.1.1 Funzionamento . . . . . 4
2.1.2 Consenso . . . . . 4
2.1.3 Classificazione delle blockchain . . . . . 6
2.1.4 Chiave pubblica e chiave privata . . . . . 7
2.2 Ethereum . . . . . 9
2.2.1 Etherem vs Bitcoin . . . . . 9
2.2.2 Funzionamento . . . . . 10
2.3 Smart Contracts . . . . . 13
2.3.1 Esempio di smart contract . . . . . 14
2.4 Ethereum Virtual Machine . . . . . 16
2.5 EVM Bytecode . . . . . 19
2.6 Alterazione del flusso di esecuzione . . . . . 21
2.6.1 Esempio di alterazione . . . . . 22
2.6.2 Pushed jumps vs Orphan jumps . . . . . 25
2.7 L'importanza di costruire un CFG completo . . . . . 26
2.8 Vulnerabilità di rientranza . . . . . 27
2.8.1 Attacco a The DAO (2016) . . . . . 29

**3 Analisi Statica** **31**

**4 EVMLiSA** **33**

**5 Conclusione** **35**

**Bibliografia** **37**

# Elenco delle figure

2.1 Funzionamento della blockchain. . . . . 4
2.2 Esempio di comunicazione utilizzando un sistema di crittografia asimmetrica [1]. . . . . 8
2.3 Il processo IFTTT negli smart contracts (MM: TODO sistemare con photoshop questa figura) . . . . . 14
2.4 Stato dello stack (a) prima e (b) dopo l'esecuzione dell'opcode `ADD`. . . . . 20
2.5 CFG dell'Algoritmo 6. . . . . 24

# Elenco degli algoritmi


1 Esempio di smart contract in Solidity. . . . . 15
2 Frammento di bytecode dell'EVM. . . . . 19
3 Traduzione in linguaggio umano dell'Algoritmo 2. . . . . 19
4 Esempio di EVM bytecode con l'opcode `JUMP`. . . . . 21
5 Esempio di EVM bytecode con l'opcode `JUMPI`. . . . . 21
6 Esempio ciclo While in EVM bytecode. . . . . 22
7 Implementazione in C dell'Algoritmo 6. . . . . 24
8 Esempio di Orphan Jump. . . . . 25
9 Vulnerabilità di rientranza in uno smart contract vittima. . . . 28
10 Smart contract che sfrutta la vulnerabilità dell'Algoritmo 9. . 29

# Capitolo 1

# Introduzione

Ethereum è una piattaforma blockchain che consente la creazione di un ambiente globale decentralizzato, in cui gli utenti possono sviluppare strumenti digitali sicuri utilizzando la valuta nativa Ether (ETH) per transazioni e servizi computazionali [2].

La rete Ethereum è costituita da nodi interconnessi che seguono regole definite nel protocollo Ethereum, supportando una vasta gamma di comunità, applicazioni, organizzazioni e attività digitali accessibili a chiunque abbia una connessione internet.

La caratteristica principale di Ethereum è la capacità di eseguire smart contract tramite la Ethereum Virtual Machine (EVM), consentendo funzionalità avanzate al di là delle transazioni finanziarie di base (e.g. Bitcoin [3]). Gli smart contract sono programmi immutabili memorizzati sulla blockchain che eseguono azioni predefinite senza intervento umano.

Tuttavia, garantire la sicurezza e l'affidabilità degli smart contract è essenziale, poiché non possono essere modificati una volta deployati. La presenza di bug o vulnerabilità potrebbe causare gravi conseguenze, come perdite di fondi o azioni indesiderate.

Per garantire la qualità degli smart contract, vengono sviluppate tecniche di analisi e verifica del codice. L'analisi statica, ad esempio, identifica potenziali problemi di sicurezza senza eseguire effettivamente il codice. Tuttavia, nel contesto di Ethereum, la costruzione di un Control Flow Graph (CFG) affidabile è complessa a causa delle destinazioni dinamiche dei salti (Jump Orfane) durante l'esecuzione del EVM bytecode.

In questo elaborato viene presentato EVMLiSA[1], un software in grado di costruire un CFG sound e affidabile (MM: TODO si spera) di smart contract eseguibili su EVM.

[1] `https://github.com/lisa-analyzer/evm-lisa`.

(MM: Scrivere la parola ether in corsivo in tutta la tesi)

# Capitolo 2

# Background

(MM: TODO scrivere riassuntino)

## 2.1 Blockchain

Il concetto di blockchain è stato introdotto per la prima volta nel 2008 da Satoshi Nakamoto con il whitepaper di Bitcoin [3] e successivamente implementato nel 2009 come tecnologia alla base di Bitcoin.

È importante sottolineare che Bitcoin e blockchain sono due entità separate: Bitcoin rappresenta solo una delle molteplici applicazioni costruite sulla tecnologia blockchain.

La blockchain è un registro digitale, decentralizzato e distribuito su una rete, strutturato come una catena di blocchi responsabili dell'archiviazione dei dati. Questi dati possono comprendere informazioni di valore economico o intere applicazioni digitali. È possibile aggiungere nuovi blocchi di informazioni, ma non è possibile modificare o rimuovere i blocchi precedentemente aggiunti alla catena.

In questo ambiente, la crittografia e i protocolli di consenso garantiscono sicurezza e immutabilità. Il risultato è un sistema aperto, neutrale, affidabile e sicuro, dove la nostra capacità di utilizzare e fidarci del sistema non dipende dalle intenzioni di nessun individuo o istituzione.

La blockchain può essere considerata parte della famiglia delle Distributed Ledger Technology (DLT): tutti i partecipanti alla rete hanno accesso al registro distribuito e al suo registro immutabile delle transazioni. Questo registro condiviso consente di registrare le transazioni una sola volta, eliminando la duplicazione degli sforzi tipica delle reti aziendali tradizionali [4].

### 2.1.1 Funzionamento

La blockchain è composta da una serie di blocchi, aggiunti uno dopo l'altro in modo sequenziale. Ogni blocco contiene una prova matematica, resa possibile dall'utilizzo della crittografia, che ne assicura la sequenzialità tramite un hash del blocco precedente.

Sebbene il concetto di catena di blocchi sia comune a quasi tutti i sistemi blockchain, la struttura e il contenuto dei blocchi possono variare a seconda dello scopo della blockchain stessa. Ad esempio, i blocchi della blockchain di Bitcoin possono essere diversi da quelli della blockchain di Ethereum.

I blocchi sono interconnessi tramite funzioni di hash crittografiche (e.g. SHA-256 (MM: TODO forse andrebbe spiegato?)), creando un legame matematico tra di loro. Questo processo fornisce un modo conveniente per esprimere l'intero contenuto della blockchain in una singola stringa di lunghezza definita.

Ogni blocco contiene informazioni specifiche (e.g. le transazioni) e l'hash del blocco precedente (Figura 2.1). Modificare qualsiasi informazione in un blocco altererebbe l'hash del blocco e, di conseguenza, tutti gli hash successivi, rendendo immediatamente evidente l'alterazione.

Ogni nuovo blocco rafforza la verifica dei blocchi precedenti, rendendo la blockchain a prova di manomissione e fornendo un registro affidabile delle transazioni su cui gli utenti possono fare affidamento [4].

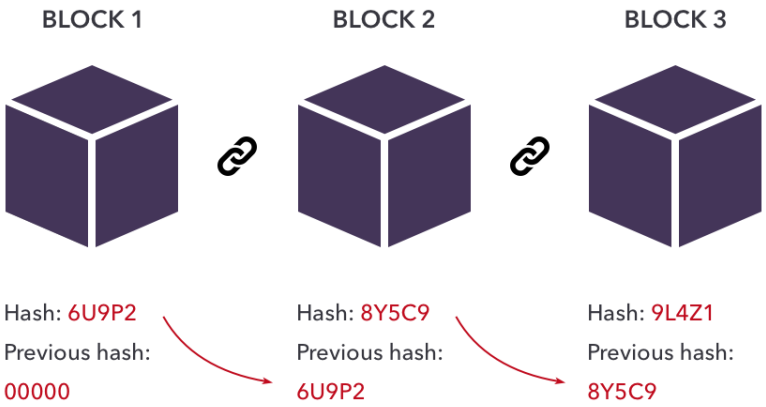


Figura 2.1: Funzionamento della blockchain.

### 2.1.2 Consenso

Il consenso è il processo mediante il quale i nodi della rete raggiungono un accordo su ciò che è avvenuto all'interno della blockchain. Questo accordo

rappresenta l'unica verità sullo stato attuale della blockchain e viene ottenuto attraverso un accordo generale tra i partecipanti della rete. Il consenso non è un evento istantaneo, ma un processo continuo che coinvolge vari partecipanti con ruoli e responsabilità specifici. Si basa su principi matematici ed e conomici per incentivare tutti i membri a raggiungere un accordo su un unico dato [4].

Nella pratica, il consenso richiede che i nuovi blocchi siano approvati dalla maggioranza dei nodi della rete, che sono distribuiti su una vasta rete di computer. Questo processo può avvenire attraverso due meccanismi principali: la Proof of Work (PoW) e la Proof of Stake (PoS).

- Proof of Work (PoW): È il sistema di consenso utilizzato principalmente da Bitcoin. Qui, i nodi della rete, noti come "miner", competono per risolvere complessi problemi crittografici. Il primo nodo che risolve con successo il problema crittografico viene premiato con una quantità specifica di criptovaluta. La difficoltà di generare l'hash viene modificata man mano che la rete si espande, in modo che i nuovi blocchi vengano creati e approvati a un ritmo costante al variare della potenza di calcolo della rete. La difficoltà di generare blocchi nella blockchain di Bitcoin, ad esempio, viene regolata modificando il numero di zeri con cui devono iniziare, garantendo che un nuovo hash venga trovato solo una volta ogni dieci minuti circa dall'intera rete. Questi problemi sono progettati per richiedere una grande quantità di potenza di calcolo e l'energia utilizzata in questo processo è sia una caratteristica distintiva che un punto di vulnerabilità di PoW. Se da un lato rende la blockchain sicura, poiché sarebbe estremamente costoso e poco pratico per un singolo attaccante possedere la maggior parte della potenza di calcolo della rete, dall'altro richiede una notevole quantità di energia elettrica [5].

- Proof of Stake (PoS): È un meccanismo di consenso in cui i nodi della rete, chiamati "validatori", competono per essere selezionati per convalidare il prossimo blocco. La probabilità di essere selezionati è proporzionale alla quantità di criptovaluta che il nodo ha depositato nella blockchain. Quando un validatore viene selezionato, "mette in gioco" (stake) i propri fondi (token). Se il validatore non adempie ai propri doveri o tenta di commettere una frode, rischia di perdere i suoi fondi. PoS non richiede una quantità significativa di energia elettrica come PoW, ma presenta un problema di monopolio della ricchezza, poiché coloro che possiedono più token hanno maggiori probabilità di essere selezionati per convalidare il blocco successivo e ricevere la ricompensa. Questo problema è mitigato con l'introduzione di Delegated Proof

of Stake (DPoS), che prevede un sistema di voto per la selezione dei validatori, rendendo il processo più democratico [5].

### 2.1.3 Classificazione delle blockchain

Le blockchain possono essere categorizzate in vari tipi in base alla loro accessibilità e ai requisiti di utilizzo:

- Permissionless: Questo tipo di blockchain permette a chiunque di accedervi senza restrizioni, senza la necessità di identificarsi. Esempi notevoli sono Bitcoin ed Ethereum. Tuttavia, questa libertà può portare a problemi di dimensionamento e sicurezza, poiché ogni nodo della rete deve elaborare e convalidare l'intera blockchain, rendendo anche più probabile l'ingresso di attori malevoli [6].

- Permissioned: Al contrario, nelle blockchain ad accesso limitato, l'accesso è controllato e richiede l'autenticazione da parte di un'autorità centrale. Questo approccio riduce i problemi di dimensionamento e sicurezza riscontrati nelle blockchain aperte. Tuttavia, l'intervento di un'autorità centrale compromette parzialmente il principio di decentralizzazione [6].

Ulteriormente, le blockchain possono essere suddivise in base alle loro applicazioni e requisiti specifici:

- Public: Le blockchain pubbliche incoraggiano la partecipazione di tutti gli utenti, offrendo incentivi come ricompense in criptovaluta. Sono trasparenti e flessibili. Esempi ben noti includono Bitcoin ed Ethereum [7].

- Private: Le blockchain private sono utilizzate all'interno di reti aziendali o organizzative, spesso per migliorare l'efficienza dei processi interni. Sono meno trasparenti rispetto alle blockchain pubbliche e richiedono solitamente un'autorizzazione per accedervi [7].

- Consortium: Queste blockchain sono progettate per la collaborazione tra più entità, come aziende o istituzioni. La gestione dell'accesso è condivisa tra i partecipanti, garantendo un certo livello di decentralizzazione senza dipendenza da un'autorità centrale [7].

### 2.1.4 Chiave pubblica e chiave privata

La tecnologia blockchain si basa su uno dei principi fondamentali della sicurezza informatica: la *crittografia*. La crittografia è l'insieme di tecniche utilizzate per rendere un messaggio incomprensibile a chi non è autorizzato a leggerlo, garantendo così la confidenzialità dei dati, un pilastro cruciale nella sicurezza informatica moderna.

Nel panorama della blockchain, si fa ampio uso della crittografia asimmetrica, conosciuta anche come *crittografia a chiave pubblica*. Questo tipo di crittografia coinvolge una coppia di chiavi per ciascun partecipante alla comunicazione:

- la chiave pubblica, accessibile a chiunque debba inviare informazioni;
- la chiave privata, custodita esclusivamente dal proprietario.

In un sistema basato sulla crittografia a chiave pubblica, qualsiasi persona può cifrare un messaggio utilizzando la chiave pubblica del destinatario, ma solo il destinatario può decifrarlo con la sua chiave privata. Questo meccanismo consente a due utenti di comunicare in modo sicuro anche su canali non sicuri, poiché la sicurezza dipende esclusivamente dalla custodia segreta della chiave privata [1].

Nel contesto della blockchain, la crittografia asimmetrica viene utilizzata per consentire lo scambio di asset digitali, come le criptovalute. Ogni partecipante alla blockchain possiede una coppia di chiavi, una pubblica e una privata.

Per comprendere meglio il funzionamento, consideriamo un esempio pratico con Alice e Bob:

- entrambi sono identificati all'interno della blockchain tramite un indirizzo pubblico (chiave pubblica), noto come *"address"*;
- chiunque desideri inviare loro degli asset deve farlo tramite il loro rispettivo indirizzo;
- ognuno di loro possiede la sua chiave privata, necessaria per autorizzare le transazioni in uscita;
- questa chiave privata è fondamentale per garantire che solo il legittimo proprietario possa effettuare transazioni.

Quando Alice vuole inviare 1 ether a Bob, accede ai suoi fondi utilizzando la sua chiave privata e invia la transazione verso l'indirizzo di Bob. La

transazione è crittografata con la chiave pubblica di Bob, garantendo la sicurezza del trasferimento. Solo Bob, utilizzando la sua chiave privata, sarà in grado di decifrare e confermare la transazione, garantendo così la legittimità dell'operazione. È fondamentale che entrambi custodiscano le proprie chiavi private in modo sicuro per evitare accessi non autorizzati [8].

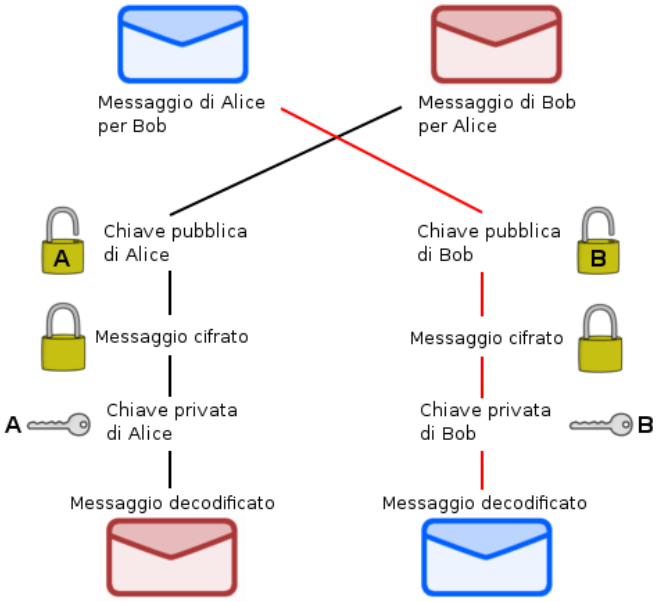


Figura 2.2: Esempio di comunicazione utilizzando un sistema di crittografia asimmetrica [1].

## 2.2 Ethereum

Ethereum è una blockchain aperta (*permissionless*), accessibile a chiunque (*pubblica*) e con il codice sorgente disponibile liberamente (*open-source*). È stata ideata per la prima volta da Vitalik Buterin nel 2013 [9], con l'obiettivo di creare una blockchain in grado di eseguire una vasta gamma di programmi generici.

Dal punto di vista tecnico, Ethereum può essere considerato una sorta di enorme macchina virtuale globale e *"infinita"*, che opera seguendo uno stato singolo accessibile da qualsiasi parte del mondo e una macchina virtuale che applica le modifiche a tale stato.

Tuttavia, in termini più pratici, Ethereum si presenta come un'infrastruttura informatica decentralizzata, aperta a tutti e basata su codice sorgente accessibile[1], che consente l'esecuzione di programmi denominati Smart Contracts (Sezione 2.3). Utilizza una blockchain per tenere traccia e registrare le variazioni di stato del sistema, utilizzando la criptovaluta nativa (chiamata Ether) per misurare e regolare i costi delle risorse di elaborazione.

Attraverso la piattaforma Ethereum, gli sviluppatori hanno la possibilità di creare applicazioni decentralizzate con funzioni economiche integrate, offrendo non solo elevata disponibilità, verificabilità, trasparenza e neutralità, ma anche la riduzione o l'eliminazione della censura e alcuni rischi associati alle controparti tradizionali [10].

### 2.2.1 Etherem vs Bitcoin

Ethereum presenta numerose caratteristiche comuni con altre blockchain aperte: una rete peer-to-peer che collega gli utenti, un algoritmo di consenso per mantenere gli aggiornamenti di stato sincronizzati (inizialmente basato su un consenso proof-of-work, ma con l'avvento dell'aggiornamento *The Merge*[2], il consenso è passato a proof-of-stake), l'uso di tecniche crittografiche come firme digitali e hash, e una valuta digitale (ETH).

Tuttavia, sia lo scopo che la struttura di Ethereum si differenziano notevolmente da quelle delle blockchain antecedenti, come Bitcoin.

Il principale obiettivo di Ethereum non è solo quello di fungere da sistema di pagamento digitale. Anche se l'ether è fondamentale per il funzionamento di Ethereum, essa viene considerata una *"valuta di utilità"* utilizzata per pagare l'utilizzo della piattaforma Ethereum come un computer globale.

[1] `https://github.com/ethereum/go-ethereum`
[2] `https://ethereum.org/it/roadmap/merge/`

A differenza di Bitcoin, che dispone di un linguaggio di scripting limitato, Ethereum è stato progettato per essere una blockchain programmabile per scopi generici, con una macchina virtuale EVM in grado di eseguire codice di complessità arbitraria e illimitata. Mentre il linguaggio di scripting di Bitcoin si limita principalmente a valutazioni semplici (true/false) delle condizioni di spesa di un utente, il linguaggio di Ethereum (Solidity[3]) è quasi completo in termini di capacità computazionale (Turing completeness), consentendo ad Ethereum di funzionare come un computer per scopi generali [10].

### 2.2.2 Funzionamento

Sulla blockchain Ethereum sistono due tipi principali di account utilizzabili per gestire le transazioni e l'interazione sulla rete: gli *Externally Owned Account* e i *Contract Account*.

Gli *Externally Owned Account* (EOA) sono controllati direttamente dagli utenti e sono associati a una coppia di chiavi crittografiche, pubblica e privata. Questi account consentono agli individui di ricevere e inviare ether e di partecipare alle transazioni sulla rete. Le transazioni tra due account esterni coinvolgono esclusivamente lo scambio di ether e non comportano alcun costo di creazione dell'account [11].

I *Contract Account*, invece, sono associati agli smart contract. Questi account contengono il codice dello smart contract e vengono attivati quando ricevono una transazione. Le transazioni con questi account possono coinvolgere l'esecuzione di codice dello smart contract, oltre allo scambio di ether e comportano un costo di creazione in quanto richiedono risorse di calcolo e di archiviazione sulla rete Ethereum [11].

Affinché una transazione venga effettuata su Ethereum, il mittente deve conoscere l'indirizzo del destinatario, detto anche chiave pubblica dell'account, e firmare digitalmente la transazione con la propria chiave privata. Questo processo dimostra che il richiedente della transazione è il legittimo proprietario dell'account.

Le transazioni sono essenzialmente istruzioni crittograficamente firmate da un account che iniziano una modifica dello stato della rete Ethereum. Inoltre, per "transazione" si intende una transazione approvata e inclusa in un blocco della blockchain.

Poiché le transazioni sono atomiche, esse devono essere eseguite completamente prima di apportare cambiamenti allo stato globale della rete. Questo significa che tutte le istruzioni all'interno della transazione devono essere valide. Se una qualsiasi istruzione fallisce, gli effetti della transazione vengo-

[3] `https://docs.soliditylang.org/en/latest/`

no annullati e lo stato viene ripristinato al momento precedente (rollback), come se la transazione non fosse mai avvenuta. Anche se una transazione fallisce, viene comunque registrata come tentata, ma non influenza lo stato complessivo della rete [12].

Ogni transazione su Ethereum comporta un costo proporzionale alla sua complessità computazionale, misurato in *"gas"*. Possiamo pensare al gas come al carburante necessario per far funzionare le operazioni sulla blockchain, simile al carburante utilizzato da un'auto per percorrere una determinata distanza.

Ogni operazione sulla blockchain ha un costo specifico in gas. Ad esempio, calcolare un hash o fare una somma di due numeri richiede una differente quantità di gas (`30` e `3`, rispettivamente).

Il gas è misurato in "wei" (1 wei $= 1^{-18}$ ETH ) ed è strettamente legato alle transazioni. Ogni transazione su Ethereum ha due parametri: il prezzo del gas (gas price) e il limite del gas (gas limit), che rappresentano rispettivamente il prezzo che si è disposti a pagare per unità di gas e la quantità massima di gas utilizzabile.

A differenza di Bitcoin, dove esiste un limite alla dimensione massima di un blocco, su Ethereum si fa riferimento al limite di gas, che determina la massima quantità di calcoli che la Ethereum Virtual Machine deve eseguire per blocco.

Immaginiamo di dover eseguire una transazione che richiede 10 gas e abbiamo deciso di pagare `100 wei` per ogni gas: il costo totale della transazione sarà di `1000 wei` ($10 \times 100$). Se aumentiamo il prezzo del gas a `1000 wei` per gas, il costo totale della transazione sarà di 10000 wei ($10 \times 1000$). I validatori della rete Ethereum sono liberi di scegliere le transazioni da includere nel nuovo blocco, e generalmente cercano di massimizzare il profitto. Di conseguenza, le transazioni con un prezzo del gas più elevato hanno una priorità maggiore di essere aggiunte al blocco successivo.

Se una transazione richiede `15` gas ma abbiamo impostato un limite di `10` gas, l'esecuzione si interromperà quando verrà raggiunto il limite, e tutto il gas andrà perso. Se, invece, il limite di gas è superiore a quello utilizzato dalla transazione, il gas in eccesso verrà restituito al mittente.

Quindi, il costo totale di una transazione può essere calcolato con la seguente formula:

$$\text{Costo transazione} = \text{Limite di gas} \times \text{Prezzo del gas}$$

Oltre a compensare i validatori, il costo delle transazioni su Ethereum serve anche a proteggere la blockchain da attacchi, come il DDoS (Distributed Denial of Service), e da errori di programmazione negli smart contract

che potrebbero sovraccaricare il sistema. Ad esempio, se una computazione finisse in un ciclo infinito, la gestione del gas interromperebbe l'esecuzione una volta che il gas disponibile è esaurito. Questa caratteristica ci consente di definire Ethereum come un linguaggio (quasi) Turing-completo, poiché siamo sicuri che ogni programma in esecuzione terminerà [4].

## 2.3 Smart Contracts

Il concetto di smart contract è stato introdotto per la prima volta da Nick Szabo nel 1996, definendolo come *"un protocollo di transazione digitale che esegue i termini di un contratto"* [13]. Lo scopo principale di uno smart contract è quello di automatizzare l'adempimento delle condizioni contrattuali, riducendo al minimo il rischio di azioni malevole e la necessità di fidarsi degli intermediari (rischio di controparte).

Un contratto, in generale, è un accordo legalmente vincolante tra due o più parti e svolge un ruolo fondamentale nell'instaurare fiducia tra i soggetti coinvolti in una transazione[4]. Può essere tanto semplice quanto un biglietto dell'autobus o molto complesso come un contratto di lavoro.

Nel contesto delle blockchain, uno smart contract è un programma che replica tutte le caratteristiche di un contratto tradizionale, ma viene memorizzato ed eseguito all'interno di una blockchain. Si tratta di un agente autonomo che risiede su una blockchain, senza la necessità di un'entità esterna per valutare le condizioni e prendere decisioni. Questo ruolo è sostituito dal consenso della rete. Gli smart contract stabiliscono le regole e le fanno rispettare automaticamente alle parti coinvolte, senza la necessità di autorità centrali. Quando le condizioni del contratto vengono soddisfatte, lo smart contract esegue autonomamente azioni specifiche, come ad esempio il trasferimento di denaro [4].

Un smart contract può essere paragonato a un'applicazione IFTTT (If This Then That) che risponde a eventi specifici (Figura 2.3).

Gli smart contract, solitamente redatti in un linguaggio di alto livello come Solidity, necessitano di essere compilati nel bytecode di basso livello eseguito nell'EVM per poter essere eseguiti. Dopo la compilazione, vengono distribuiti sulla piattaforma Ethereum attraverso una speciale transazione di creazione del contratto, inviata all'indirizzo di creazione del contratto `0x0`, detto *zero address*. Ogni contratto è identificato da un indirizzo Ethereum, derivato dalla transazione di creazione del contratto in funzione dell'account di origine e del nonce. Come già menzionato nella sezione 2.2.2, a differenza degli EOAs, gli account creati per nuovi smart contract non hanno chiavi associate.

Gli smart contract vengono eseguiti solo quando vengono richiamati da una transazione. Ogni smart contract in Ethereum viene eseguito a seguito di una transazione avviata da un EOA. Un contratto può richiamare un altro contratto, che a sua volta può richiamare un altro contratto e così via, ma il

[4] `https://www.treccani.it/enciclopedia/contratto/`

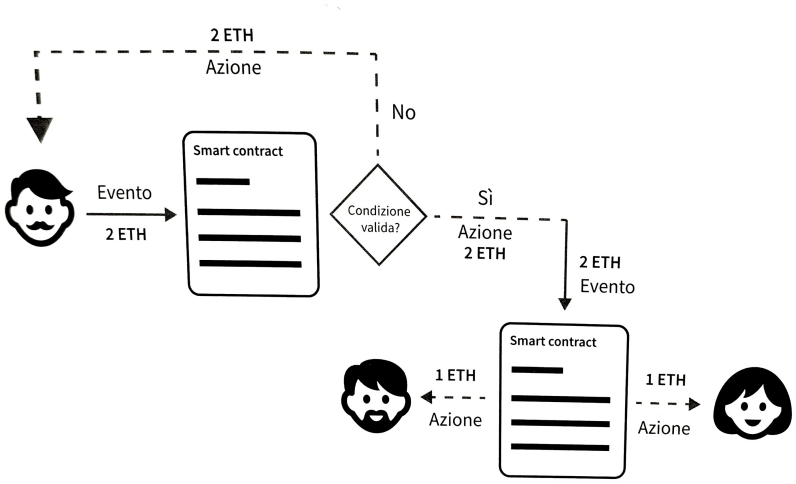


Figura 2.3: Il processo IFTTT negli smart contracts
(MM: TODO sistemare con photoshop questa figura)

primo contratto in questa catena di esecuzione sarà sempre stato richiamato da una transazione proveniente da un EOA.

Le transazioni sono atomiche, indipendentemente dal numero di smart contract richiamati o dalle azioni che eseguono. Ogni transazione viene eseguita nella sua totalità e le eventuali modifiche allo stato globale vengono registrate solo se l'esecuzione termina con successo. Se l'esecuzione fallisce a causa di un errore, tutti gli effetti vengono annullati e la transazione viene registrata come tentata.

È importante notare che una volta distribuito, il codice di un contratto non può essere modificato. Approfondiremo ulteriormente questo aspetto in seguito.

### 2.3.1 Esempio di smart contract

Ma adesso vediamo un esempio di smart contract[5] nell'Algoritmo 1.

Questo smart contract è composto da due contratti: *ReceiveEther* e *SendEther*.

Il contratto *ReceiveEther* è progettato per ricevere *ether*. Contiene due funzioni: `receive` e `fallback`. La funzione `receive` viene chiamata quando viene inviato ether al contratto e non ci sono dati associati alla transazione. Questo è possibile tramite l'invio di ether direttamente all'indirizzo del contratto senza specificare alcuna funzione. La funzione `fallback` viene invece

[5]Tratto da: `https://solidity-by-example.org/sending-ether/`

chiamata quando viene inviato ether al contratto e sono presenti dati associati alla transazione (e.g. (MM: TODO fare esempio)). Entrambe le funzioni accettano ether e sono contrassegnate come `payable` per indicare che possono ricevere fondi. È importante specificare che un contratto che riceve ether deve avere almeno una di queste due funzioni. La funzione `getBalance` restituisce il saldo attuale del contratto in ether.

Il contratto *SendEther* è progettato invece per inviare ether. Contiene una sola funzione `sendViaCall` che invia ether a un altro indirizzo Ethereum. Questa funzione prende in input l'indirizzo del destinatario e l'ammontare di ether da inviare. Utilizza la funzione `call` per inviare ether al destinatario e restituisce un booleano che indica se l'operazione è stata eseguita con successo. Infine, la funzione `require` permette di verificare che l'invio sia andato a buon fine, in caso contrario l'esecuzione viene interrotta con un messaggio di errore.

**Algoritmo 1** Esempio di smart contract in Solidity.

```
pragma solidity ^0.8.24;

contract ReceiveEther {
    receive() external payable {}

    fallback() external payable {}

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

contract SendEther {
    function sendViaCall(address payable _to) public payable {
        (bool sent, bytes memory data) = _to.call{value: msg.value}("");
        require(sent, "Failed to send Ether");
    }
}

```

## 2.4 Ethereum Virtual Machine

Al cuore del protocollo e del funzionamento di Ethereum risiede l'Ethereum Virtual Machine (EVM). Questa componente software gestisce l'implementazione e l'esecuzione degli smart contracts. Quasi ogni azione sulla rete Ethereum comporta un aggiornamento dello stato calcolato dall'EVM, ad eccezione delle semplici transazioni di trasferimento di valore da un EOA ad un altro. A un livello più alto, possiamo concepire l'EVM come un enorme computer decentralizzato, distribuito su scala globale, che contiene milioni di "programmi" eseguibili, ognuno dei quali con il proprio spazio dati permanente [10].

A livello matematico, possiamo definire l'EVM come una funzione matematica: dato un input, essa produce un output deterministico. In particolare, la funzione di transizione è definita come segue:

$$Y(S, T) = S'$$

dove $S$ è il vecchio stato macchina valido e $T$ è un insieme di transazioni valide che, se applicate allo stato $S$, producono lo stato $S'$ [14].

Come detto in precedenza, l'EVM è una macchina quasi Turing-completa, il che significa che tutti i processi di esecuzione sono limitati a un numero finito di passaggi computazionali, determinati dalla quantità di gas disponibile per ogni esecuzione di uno smart contract. Questo limite assicura che ogni programma abbia una fine definita, evitando situazioni in cui l'esecuzione potrebbe protrarsi all'infinito, portando alla congestione dell'intera piattaforma Ethereum [10].

Dal punto di vista architetturale, l'EVM utilizza uno stack, ovvero una coda volatile di tipo LIFO (Last In First Out), per memorizzare tutti i valori in memoria, con una profondità massima di 1024 elementi. Opera su parole (words) di 256 bit, principalmente per agevolare le operazioni di hashing, e include diverse componenti di dati indirizzabili:

- una ROM virtuale contenente il codice del programma, immutabile una volta caricato il bytecode dello smart contract da eseguire;
- una memoria volatile (Memory), in cui ogni posizione è inizializzata a zero e può essere utilizzata temporaneamente durante l'esecuzione;
- uno spazio di archiviazione permanente (Storage), parte dello stato di Ethereum, anch'esso inizializzato a zero, che conserva informazioni a lungo termine.

Il ruolo principale dell'EVM consiste nell'aggiornare lo stato di Ethereum attraverso il calcolo di transizioni di stato valide derivanti dall'esecuzione del codice degli smart contracts. Questo concetto rende Ethereum una macchina a stati basata sulle transazioni, poiché gli attori esterni, come gli utenti della rete, iniziano le transizioni di stato creando, accettando e ordinando le transazioni.

Per comprendere meglio cosa costituisca lo stato di Ethereum, possiamo suddividerlo in due livelli. Al livello superiore troviamo lo stato globale di Ethereum, che è essenzialmente una mappatura degli indirizzi Ethereum (160 bit) agli account della rete [10].

Scendendo al livello inferiore, ogni indirizzo Ethereum rappresenta un account, che include diverse informazioni:

- il saldo in ether, espresso come il numero di wei posseduti dall'account;
- il nonce, che tiene traccia del numero di transazioni inviate con successo da quell'account, se si tratta di un account controllato dall'utente (EOA), o il numero di smart contract creati, se è un contract account;
- lo storage dell'account, che funge da archivio dati permanente utilizzato esclusivamente dagli smart contracts;
- il codice del programma dell'account, presente solo se l'account è uno smart contract. Da notare che un account esterno controllato dall'utente avrà sempre codice nullo e uno storage vuoto.

Quando una transazione coinvolge l'esecuzione del codice di uno smart contract, viene creata un'istanza dell'EVM (Ethereum Virtual Machine) con tutte le informazioni necessarie relative al blocco corrente in fase di creazione e alla transazione specifica in elaborazione. In questo processo, il codice dello smart contract viene caricato nell'EVM, il program counter viene impostato a zero, lo storage del contract account viene recuperato, la memoria viene inizializzata e le variabili di blocco e di ambiente vengono configurate.

Un aspetto fondamentale è la quantità di gas disponibile per questa esecuzione, determinata dalla quantità di gas pagata dal mittente all'inizio della transazione. Durante l'esecuzione del codice, la quantità di gas disponibile diminuisce in base al costo delle operazioni eseguite. Se il gas disponibile si esaurisce, viene lanciata un'eccezione "Out of Gas (OOG)" e l'esecuzione viene interrotta, annullando la transazione. Nessuna modifica viene apportata allo stato di Ethereum, tranne l'incremento del nonce del mittente e il pagamento delle commissioni al validatore.

Possiamo immaginare l'EVM che opera su una copia isolata dello stato effettivo di Ethereum, scartando completamente questa versione isolata se l'esecuzione non può essere completata. Tuttavia, se l'esecuzione ha successo, lo stato effettivo viene aggiornato per corrispondere alla versione isolata, inclusi eventuali cambiamenti nei dati di archiviazione del contratto chiamato, la creazione di nuovi smart contracts e i trasferimenti di ether avviati [10].

## 2.5 EVM Bytecode

Come accennato in precedenza, Ethereum esegue gli smart contract all'interno dell'Ethereum Virtual Machine (EVM), un ambiente di runtime decentralizzato che si occupa dell'esecuzione del codice di tali contratti. Gli sviluppatori possono scrivere gli smart contract utilizzando diversi linguaggi di alto livello, come Solidity[6] o Vyper[7], ma affinché possano essere eseguiti dall'EVM, devono essere compilati in un linguaggio di basso livello noto come EVM bytecode. Il bytecode dell'EVM è un linguaggio a basso livello basato su stack che comprende circa 150 istruzioni chiamate opcodes [8], le quali vengono interpretate dall'EVM per manipolare uno stack i cui elementi sono parole di 256 bit. Ogni istruzione è rappresentata da un numero esadecimale preceduto da `0x`.

Consideriamo questo breve frammento di bytecode dell'EVM:

**Algoritmo 2** Frammento di bytecode dell'EVM.

```
60 01 60 02 01
```

Il byte `60` corrisponde all'opcode `PUSH1`, il quale inserisce un byte nello stack. Il byte successivo è `01`, che corrisponde al valore che e' stato messo in cima allo stack. Allo stesso modo, i byte `60 02` rappresentano l'istruzione `PUSH1 0x02` (lo stack dopo l'esecuzione di questi opcode è mostrato in Figura 2.4a). L'ultimo byte, ossia `01`, corrisponde all'opcode `ADD`, il quale preleva due elementi dallo stack, li somma e mette il risultato di nuovo sulla pila. Pertanto, la versione tradotta in linguaggio umano della stringa di bytecode appena analizzata è:

**Algoritmo 3** Traduzione in linguaggio umano dell'Algoritmo 2.

```
PUSH1 0x01
PUSH1 0x02
ADD
```

Dopo l'esecuzione di queste istruzioni, l'elemento in cima allo stack avrà il valore `3` (lo stack dopo l'operazione `ADD` è mostrato nella Figura 2.4b).

[6] `https://docs.soliditylang.org/en/latest/`
[7] `https://docs.vyperlang.org/en/stable/toctree.html`
[8] Lista completa: `https://ethereum.github.io/yellowpaper/paper.pdf`

Il set di istruzioni dell'EVM include una vasta gamma di operazioni, tra cui:

- operazioni aritmetiche e logiche bit a bit;
- richieste del contesto di esecuzione;
- accesso allo stack, alla memoria e allo storage;
- controllo del flusso di esecuzione;
- logging, calling e altre operazioni.

Oltre alle operazioni standard del bytecode, l'EVM offre anche accesso a informazioni specifiche sull'account (come indirizzo e saldo) e sul blocco (come numero di blocco e prezzo attuale del gas) [10].

Questo elaborato si concentrerà sull'analisi del controllo del flusso di esecuzione.

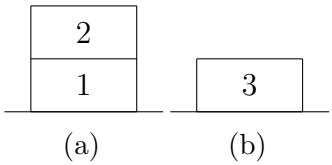


Figura 2.4: Stato dello stack (a) prima e (b) dopo l'esecuzione dell'opcode `ADD`.

## 2.6 Alterazione del flusso di esecuzione

Il flusso di esecuzione di uno smart contract (in EVM bytecode) inizia con il primo opcode e procede in sequenza. Gli unici opcodes che possono alterare il flusso di esecuzione di uno smart contract senza interromperlo[9] sono `JUMP` e `JUMPI`.

L'istruzione `JUMP` comporta un salto incondizionato a una posizione specifica del programma, la quale è definita dall'indirizzo memorizzato nell'elemento più in alto dello stack (quindi il valore che viene estratto). Per semplificare, consideriamo il seguente frammento di codice:

**Algoritmo 4** Esempio di EVM bytecode con l'opcode `JUMP`.

```
PUSH1 0x01
PUSH1 0x02
JUMP
```

Quando l'istruzione `JUMP` viene eseguita, viene prelevato l'elemento in cima allo stack (in questo caso `0x20`) e il programma viene instradato verso l'istruzione all'indirizzo `0x20`, da cui procederà l'esecuzione.

D'altra parte, l'istruzione `JUMPI` rappresenta un salto condizionato: se il codice operativo `JUMPI` viene soddisfatto, l'esecuzione salta all'indirizzo memorizzato nell'elemento più in alto dello stack solo se l'elemento sottostante (cioè il secondo elemento più in alto) è diverso da zero. In caso contrario, l'esecuzione continua con l'opcode successivo. In entrambi i casi, i due elementi più in alto dello stack vengono estratti. Adesso consideriamo quest'altro frammento di codice:

**Algoritmo 5** Esempio di EVM bytecode con l'opcode `JUMPI`.

```
PUSH1 0x01
PUSH1 0x02
JUMPI
```

[9] L'esecuzione di uno smart contract può terminare in vari modi: (a) in modo implicito, quando il program counter supera l'ultimo opcode del programma; (b) in modo esplicito, durante l'elaborazione di opcodes come `STOP`, `RETURN`, `REVERT`, `SELFDESTRUCT`, `INVALID`; o (c) in modo eccezionale, quando si verificano condizioni illegali, come ad esempio lo stack underflow.

In questo scenario, la prima istruzione `PUSH1` inserisce il valore `0x10` nello stack, mentre la seconda istruzione `PUSH1` inserisce il valore `0x20`. Successivamente, l'istruzione `JUMPI` valuta il secondo elemento più alto nello stack (che è `0x10`). Poiché questo valore non è zero, l'esecuzione salta all'indirizzo memorizzato nel primo elemento dello stack (che è `0x20`).

È importante notare che la destinazione del salto, ossia l'elemento più in alto nello stack, deve corrispondere all'opcode `JUMPDEST`, altrimenti l'esecuzione terminerà in modo eccezionale. L'istruzione `JUMPDEST` non modifica lo stack; serve semplicemente a segnalare le posizioni del programma in cui è consentito un salto (condizionato o incondizionato).

### 2.6.1 Esempio di alterazione

Immaginiamo di voler creare un semplice ciclo utilizzando le istruzioni `JUMP`, `JUMPI` e `JUMPDEST` (Algoritmo 6).

**Algoritmo 6** Esempio ciclo While in EVM bytecode.

```
0      PUSH1 0x00
2      JUMPDEST
3      PUSH1 0x03
5      DUP2
6      LT
7      PUSH1 0x0D
9      JUMPI
0A     PUSH1 0x14
0C     JUMP
0D     JUMPDEST
0E     PUSH1 0x01
10     ADD
11     PUSH1 0x02
13     JUMP
14     JUMPDEST
15     STOP
```

Esaminiamo ora passo dopo passo l'esecuzione dell'Algoritmo 6:

1. `PUSH1 0x00`: Questa istruzione inserisce `0x00` nello stack.
2. `JUMPDEST`: Contrassegna una destinazione di salto.
3. `PUSH1 0x03`: Inserisce `0x03` nello stack.

4. `DUP2`: Duplica il secondo valore più alto nello stack (in questo caso, `0x00`).
5. `LT`: Confronta i due valori più alti nello stack. Se il valore superiore è inferiore al secondo, il risultato è `1`; altrimenti è `0` (in questo caso, è `1`).
6. `PUSH1 0x0D`: Inserisce `0x0D` nello stack.
7. `JUMPI`: Se il secondo valore più alto nello stack è diverso da zero, passa al blocco di codice all'indirizzo in cima allo stack (in questo caso, salta a `0x0D`, che è un salto condizionato).
8. Ora il nostro Program Counter (PC) è impostato su `0x0D`. L'operazione successiva è una `PUSH`.
9. `PUSH1 0x01`: Inserisce `0x01` nello stack.
10. `ADD`: Aggiunge i due valori più alti nello stack e mette il risultato (in questo caso, il risultato è `3`).
11. `PUSH1 0x02`: Inserisce `0x02` nello stack.
12. `JUMP`: Salta al blocco di codice all'indirizzo `0x02` e inizia la seconda iterazione del ciclo.

(MM: TODO: aggiungere due figure che mi rappresentano gli stati dello stack)

Il programma inizia un ciclo in cui ad ogni iterazione controlla una condizione `LT`. Se la condizione è vera, procede lungo un percorso specifico; altrimenti, segue un percorso alternativo. Ciò che lo rende interessante è che la condizione cambia dopo la seconda iterazione: nelle prime due iterazioni, il percorso seguito è determinato dalla condizione vera, mentre nella terza iterazione la condizione diventa falsa, portando il programma lungo un altro percorso.

Questo comportamento può essere visualizzato con la Figura 2.5, la quale rappresenta il Control Flow Graph (CFG) relativo al programma. Il CFG mostra tutti i possibili percorsi che il programma può intraprendere, insieme alle condizioni che determinano quale percorso seguire. In questo caso, il CFG ha un ciclo che rappresenta l'esecuzione ripetuta del programma, e due rami che rappresentano i due possibili percorsi che il programma può intraprendere a seconda della condizione `LT`.

L'Algoritmo 6 può essere rappresentato in C con l'Algoritmo 7.

**Algoritmo 7** Implementazione in C dell'Algoritmo 6.

```
int x = 0;
while (x < 3) {
    x++;
}
return;
```

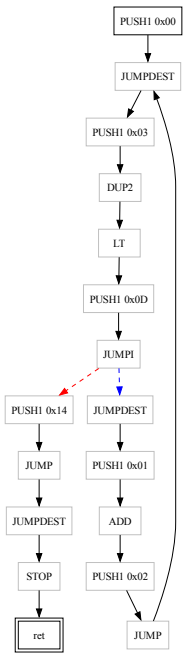


Figura 2.5: CFG dell'Algoritmo 6.

### 2.6.2 Pushed jumps vs Orphan jumps

Dopo aver spiegato come nell'EVM bytecode si possa influenzare il flusso di esecuzione attraverso gli opcodes `JUMP` e `JUMPI`, diventa evidente che la destinazione di questi salti non è staticamente definita come i dati nelle istruzioni `PUSH`, ma bensì viene calcolata dinamicamente considerando gli elementi presenti nello stack.

Tuttavia, ci sono situazioni in cui è possibile determinare staticamente la destinazione di un salto senza dover eseguire effettivamente lo smart contract. Ad esempio, nei due frammenti di codice esaminati in precedenza, le destinazioni dei salti possono essere dedotte dal codice sorgente poiché entrambi gli opcodes sono preceduti da un'istruzione `PUSH`. Questi salti sono definiti *"Pushed Jumps"* [15]. Essi non generano complicazioni nella creazione del CFG poiché le destinazioni dei salti possono essere determinate sintatticamente.

Ciononostante, una categoria di salti più complessa da gestire è rappresentata dalle *"Orphan Jumps"* [15]. Un esempio di jump orfana è il seguente:

**Algoritmo 8** Esempio di Orphan Jump.

```
PUSH1 0x0A
PUSH1 0x0C
ADD
JUMP
```

In questo caso, la destinazione dell'istruzione `JUMP` non può essere immediatamente individuata tramite un'analisi sintattica del codice sorgente; per gestire correttamente il salto e costruire un CFG accurato, è necessaria un'analisi del programma in grado di dedurre il contenuto potenziale dello stack durante l'esecuzione. Va notato che l'analisi delle jump orfane è molto complessa e un'implementazione inaccurata dello smart contract potrebbe essere sfruttata da malintenzionati per attacchi di rientranza (questo problema verrà approfondito nella Sezione 2.8).

## 2.7 L'importanza di costruire un CFG completo

Garantire la sicurezza e l'affidabilità degli smart contract è di vitale importanza per il successo della piattaforma Ethereum. Come già detto in precedenza, una volta implementati, questi contratti non possono essere modificati, il che significa che eventuali bug o vulnerabilità presenti nel codice possono causare conseguenze irrevocabili, come la perdita di fondi o l'esecuzione di azioni indesiderate. Pertanto, è fondamentale assicurarsi che gli smart contract siano privi di difetti prima della loro implementazione.

Per raggiungere questo obiettivo, i ricercatori hanno sviluppato diverse tecniche per analizzare e verificare il codice degli smart contract. Una tecnica possibile è l'analisi statica, che esamina il codice senza effettivamente eseguirlo. L'analisi statica si basa spesso sui control flow graph (CFG) per identificare potenziali problemi di sicurezza.

Tuttavia, nel contesto di Ethereum, costruire un CFG accurato e preciso per l'EVM bytecode non è banale. Questo perché alcune destinazioni dei salti sono determinate durante l'esecuzione e non durante la compilazione. Di conseguenza, non è sempre semplice costruire un CFG completo e preciso, che includa tutti i possibili percorsi di esecuzione.

Se non si riesce a costruire un CFG completo e preciso che includa tutti i possibili percorsi di esecuzione e risolva tutte le jump, si potrebbero lasciare aperture per vulnerabilità nel codice.

## 2.8 Vulnerabilità di rientranza

Gli smart contract, essendo programmi eseguiti sulla blockchain, possono essere soggetti a vulnerabilità, proprio come qualsiasi altro software. Tuttavia, le vulnerabilità negli smart contract sono particolarmente rischiose, poiché, come già spiegato nella Sezione 2.3, una volta che il contratto viene pubblicato sulla blockchain, non può essere più modificato. Tra le vulnerabilità più comuni individuate nella documentazione di Ethereum possiamo trovare la rientranza, l'overflow/underflow di interi e la manipolazione degli oracoli [5]. In questo elaborato, ci concentreremo sulla vulnerabilità di rientranza.

Per spiegare la rientranza, è importante notare che la EVM non supporta la concorrenza, il che significa che due contratti coinvolti in una chiamata di messaggio non possono essere eseguiti contemporaneamente. Quando viene effettuata una chiamata esterna (`msg.call`), l'esecuzione e la memoria del contratto chiamante vengono sospese fino al completamento della chiamata. Successivamente, l'esecuzione riprende normalmente. Questo processo può essere formalmente descritto come il trasferimento del flusso di controllo a un altro contratto.

Sebbene in gran parte innocuo, il trasferimento del flusso di controllo a contratti non fidati può causare problemi, come nel caso della rientranza. Un attacco di rientranza si verifica quando un contratto malizioso viene chiamato all'interno di un contratto vulnerabile prima che l'invocazione della funzione originale sia completata. Questo tipo di attacco può essere meglio spiegato con il seguente esempio.

Supponiamo di avere uno smart contract `Victim` che consente a chiunque di depositare e prelevare ether (Algoritmo 9)[10].

Il contratto in questione offre una funzione chiamata `withdraw()`, che consente agli utenti di prelevare gli ETH precedentemente depositati nel contratto. Durante un prelievo, il contratto esegue diverse operazioni:

1. verifica il saldo di ETH dell'utente;
2. invia i fondi all'indirizzo del chiamante;
3. azzerà il saldo dell'utente, impedendo ulteriori prelievi.

Inizialmente, verifica se le condizioni necessarie sono soddisfatte (ad esempio, se l'utente ha un saldo di `ETH` positivo). Successivamente, esegue l'interazione inviando gli `ETH` al chiamante prima di applicare gli effetti della transazione, come la riduzione del saldo dell'utente.

[10]Tratto da: `https://ethereum.org/it/developers/docs/smart-contracts/security/`

Quando `withdraw()` è chiamato da un utente (EOA), la funzione si comporta come previsto: `msg.sender.call.value()` trasferisce gli `ETH` al chiamante. Tuttavia, se `msg.sender` è uno smart contract e chiama `withdraw()`, l'invio dei fondi tramite `msg.sender.call.value()` attiverà anche l'esecuzione del codice archiviato a quell'indirizzo.

**Algoritmo 9** Vulnerabilità di rientranza in uno smart contract vittima.

```
contract Victim {
    mapping (address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external {
        uint256 amount = balances[msg.sender];
        (bool success, ) = msg.sender.call.value(amount)
    ("");
        require(success);
        balances[msg.sender] = 0;
    }
}
```

Supponiamo adesso di avere uno smart contract `Attacker` che vuole sfruttare la vulnerabilità di `Victim` (Algoritmo 10).

Questo contratto è progettato per eseguire tre operazioni:

1. accettare un deposito da un altro account (probabilmente l'EOA dell'utente malevolo);

2. depositare `1 ETH` nel contratto `Victim`;

3. prelevare `1 ETH` archiviato nel contratto `Victim`.

In apparenza, tutto sembra regolare, tranne per un dettaglio: l'`Attacker` ha incluso una funzione che richiama nuovamente `withdraw()` in `Victim` se il gas rimanente da `msg.sender.call.value` in entrata è superiore a `40.000`. Questo consente all'`Attacker` di rientrare nel contratto `Victim` e prelevare ulteriori fondi, prima che la prima invocazione di `withdraw()` di `Victim` sia completata.

Ciò accade perché in Solidity sono presenti le *funzioni di fallback*, dette anche *funzioni di default*, le quali sono definite con la forma `function()` e vengono invocate quando un contratto riceve una transazione senza specificare alcuna funzione. Il contratto `Attacker` è progettato per accettare fondi in arrivo da un'altra fonte. Quando riceve una transazione di `ETH`, viene automaticamente invocata la funzione di fallback (`function()`).

**Algoritmo 10** Smart contract che sfrutta la vulnerabilità dell'Algoritmo 9.

```
contract Attacker {
    function beginAttack() external payable {
        Victim(victim_address).deposit.value(1 ether)();
        Victim(victim_address).withdraw();
    }

    function() external payable {
        if (gasleft() > 40000) {
            Victim(victim_address).withdraw();
        }
    }
}
```

In sintesi, poiché il saldo del chiamante non viene azzerato fino al termine dell'esecuzione della funzione, le successive invocazioni avranno successo, permettendo al chiamante di prelevare ripetutamente il proprio saldo [16]. Gli attacchi di rientranza rimangono ancora oggi una grave preoccupazione per gli smart contract, come dimostrato dalle pubbliche segnalazioni di exploit di rientranza[11].

### 2.8.1 Attacco a The DAO (2016)

Quando si affronta il tema delle vulnerabilità di rientranza, è inevitabile menzionare l'attacco a The DAO del 2016, il quale rappresenta uno degli episodi più celebri nella storia di Ethereum.

The DAO era una startup che gestiva un fondo d'investimento in Ether (ETH) ed operava come uno smart contract su Ethereum. Il nome "The DAO" venne scelto dai fondatori come riferimento al concetto di Organizzazione Autonoma Decentralizzata (DAO). The DAO era un contratto che consentiva agli utenti di depositare Ether e ricevere in cambio dei token,

[11] `https://github.com/pcaversaccio/reentrancy-attacks`

utilizzabili per votare i progetti da finanziare. La sua popolarità crebbe rapidamente, diventando il più grande crowdfunding della storia (fino ad allora), con oltre 150 milioni di dollari in Ether raccolti da oltre 10.000 investitori [17].

Fin dall'inizio, il fondo si impegnò ad operare in conformità con i termini e le condizioni del proprio smart contract, che altro non erano che il codice di un programma deployato sulla blockchain.

Tuttavia, The DAO divenne famosa a causa di una vulnerabilità all'interno del proprio programma che permise a un utente sconosciuto di prelevare un terzo dei fondi. La perdita ammontò a 3,6 milioni di Ether, equivalenti a circa 60 milioni di dollari all'epoca (circa 12 miliardi di dollari ai prezzi attuali). Data l'ampia partecipazione e le implicazioni negative, Ethereum si trovò sotto forte pressione pubblica e i leader della rete decisero di eseguire un *hard-fork retroattivo* (rollback) della blockchain per annullare l'attacco.

Attraverso l'hard-fork, i fondi di The DAO furono trasferiti su un indirizzo di recupero, cancellando virtualmente la fuga di fondi. Successivamente, gli utenti del fondo poterono reclamare i propri investimenti. Ciononostante, l'hard-fork generò un significativo dissenso tra gli utenti della rete Ethereum, con alcuni dei quali che continuarono ad utilizzare la blockchain originale di Ethereum, ora denominata Ethereum Classic[12] (ETC). Tale rete opera ancora oggi, mantenendo la catena di blocchi originale, con l'utente sconosciuto che possiede ancora i fondi sottratti [18].

[12] `https://ethereumclassic.org/`

# Capitolo 3

# Analisi Statica

# Capitolo 4

# EVMLiSA

# Capitolo 5

# Conclusione

Conclusione che riassume il lavoro svolto ed eventuali lavori futuri.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

# Bibliografia


[1] Wikipedia, "Crittografia asimmetrica," 2024. [Online]. Available: https://it.wikipedia.org/wiki/Crittografia_asimmetrica

[2] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[3] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[4] G. Chiap, "Blockchain: tecnologia e applicazioni per il business," 2019.

[5] D. Tarpini, "Risoluzione di jump orfane in bytecode evm tramite stack simbolici e interpretazione astratta," 2023.

[6] Permisison.io, "Permissioned vs. permissionless blockchains explained," 2021. [Online]. Available: https://www.permission.io/blog/permissioned-vs-permissionless-blockchains-explained

[7] T. K. Sharma, "Types of blockchains explained - public vs. private vs. consortium," 2023. [Online]. Available: https://www.blockchain-council.org/blockchain/types-of-blockchains-explained-public-vs-private-vs-consortium/

[8] criptoinvestire.com, "Crittografia: la sicurezza delle blockchain," 2018. [Online]. Available: https://www.criptoinvestire.com/come-funziona-la-crittografia-nelle-blockchain.html#:~:text=La%20blockchain%20utilizza%20la%20crittografia,e%20di%20una%20chiave%20privata

[9] V. Buterin, "Ethereum white paper," *GitHub repository*, vol. 1, pp. 22–23, 2013.

[10] A. M. Antonopoulos and G. Wood, *Mastering ethereum: building smart contracts and dapps*. O'reilly Media, 2018.

[11] Nico *et al.*, "Ethereum accounts," 2023. [Online]. Available: https://ethereum.org/en/developers/docs/accounts/

[12] ——, "Ethereum smart contracts," 2023. [Online]. Available: https://ethereum.org/en/developers/docs/smart-contracts/

[13] N. Szabo, "Smart contracts: building blocks for digital markets," *EXTROPY: The Journal of Transhumanist Thought,(16)*, vol. 18, no. 2, p. 28, 1996.

[14] Nico *et al.*, "Ethereum virtual machine," 2023. [Online]. Available: https://ethereum.org/en/developers/docs/evm/

[15] M. Pasqua, A. Benini, F. Contro, M. Crosara, M. Dalla Preda, and M. Ceccato, "Enhancing ethereum smart-contracts static analysis by computing a precise control-flow graph of ethereum bytecode," *Journal of Systems and Software*, vol. 200, p. 111653, 2023.

[16] Herbie *et al.*, "Smart contract security," 2023. [Online]. Available: https://ethereum.org/it/developers/docs/smart-contracts/security/

[17] D. Siegel, "Understanding the dao attack," 2023. [Online]. Available: https://www.coindesk.com/learn/understanding-the-dao-attack/

[18] K. Olwksii, "Takeaways: 5 years after the dao crisis and ethereum hard fork," 2021. [Online]. Available: https://it.cointelegraph.com/news/takeaways-5-years-after-the-dao-crisis-and-ethereum-hard-fork

# Ringraziamenti